

Cargills Data Engineering Assessment

Architecture, Design Decisions & Implementation Rationale

Purpose

This document explains the main engineering decisions made for the Cargills Data Engineering Assessment, the reasoning behind those decisions, the assumptions made where the brief was ambiguous, and the trade-offs that would matter in a production implementation. It is intended to accompany the submitted solution and provide a concise basis for technical discussion.

1. Executive Summary

The solution uses Microsoft Fabric with a Lakehouse-centered Bronze → Silver → Gold architecture. A reusable master pipeline receives the source filename as a pipeline parameter and ingests each CSV directly from the public GitHub source through a Copy Activity. Bronze preserves the incoming data and adds source/run lineage. PySpark notebooks perform controlled schema validation, data-quality checks, rejection/quarantine, deduplication and current-state MERGE processing in Silver. Gold is built from the authoritative current Silver state and is consumed through the Fabric SQL Analytics Endpoint by Power BI.

Row ID is used as the business key because it uniquely identifies the supplied sales records and is more reliable than Order ID or Order ID + Product ID for this dataset. Later batches can therefore insert new records and update existing records without leaving stale versions in the reporting state. SQL row-level security is implemented at the data layer using a user-to-region mapping table, so the regional restriction applies to direct SQL access as well as Power BI.

2. End-to-End Architecture

Layer / Component	Decision
Source	Three supplied CSV files; each file is treated as a separate arriving batch.
Ingestion	Fabric Pipeline + Copy Activity; source filename is parameterized; binary landing preserves the file as received.
Bronze	Raw file landing plus Delta table representation with source_file_name, load_run_id and ingestion_timestamp.
Silver	PySpark validation, type standardization, record-level quarantine, batch-level schema validation, deterministic deduplication and MERGE by Row ID.
Gold	Curated fact/dimension model built from current Silver state; geography uses a deterministic geo_key.
Security	SQL RLS uses USER_NAME() against security_user_region to filter gold_sales_fact by authorized region.
Reporting	Power BI uses the secured Gold SQL endpoint; report covers sales, profit and regional performance.
Observability	Bronze/Silver processing logs capture run, batch, row counts, status and failure reason; validation failure triggers an email alert.

3. Component Choices and Trade-offs

3.1 Why Fabric Lakehouse + Pipeline + PySpark

The assessment explicitly prefers Fabric and requires reliable batch ingestion, layered storage, data-quality validation, repeatable processing, a curated reporting layer, data-layer RLS and an end to end master pipeline. The selected components keep these responsibilities separate, Pipeline handles orchestration and data movement, while Spark handles code-oriented transformation and validation.

Why PySpark instead of Dataflows Gen2

Dataflows Gen2 was considered because it provides a lower-code transformation experience. PySpark was selected because this solution needs explicit duplicate inspection, deterministic deduplication, conditional validation, rejection handling, Delta MERGE/upsert behavior and metadata-aware processing. The trade off is more code and greater Spark expertise in exchange for control and extensibility.

Why binary ingestion before parsing

The initial Copy Activity is deliberately a raw landing step. It moves the source file without parsing or changing its content. The notebook parses the CSV downstream. This preserves source fidelity and keeps ingestion separate from transformation.

Why DirectQuery instead of Import / Direct Lake

The reporting model uses DirectQuery to the Fabric SQL Analytics Endpoint because the assessment specifically requires regional security at the data layer and asks for the same restriction to hold when a user queries the data directly through SQL. The SQL endpoint is therefore the security boundary and Power BI consumes the secured Gold data. Import would introduce a separate in-memory copy that would need its own refresh/security considerations; Direct Lake was also not selected because the assessment's direct-SQL RLS requirement is central to the design.

4. Incremental Processing, Idempotency and Corrections

Business key: Row ID

Row ID is the selected business key for an individual sales record. Profiling of the supplied batches showed that Row ID uniquely identifies the records in Batch 1 and remains stable across the corrected records appearing in Batch 2. Order ID is not sufficiently unique, and Order ID + Product ID also produced duplicate combinations in the supplied data. Row ID therefore provides the strongest available record identity for the assessment.

Duplicate handling

Within an incoming batch, valid records are deduplicated by Row ID before promotion. Across batches, Silver uses Delta MERGE semantics on Row ID: a new key is inserted and an existing key is updated with the incoming valid record. This means a corrected Batch 2 record replaces the stale current version rather than creating a second reporting row.

Batch identity and reruns

The supplied source filename is treated as the logical batch identifier and is passed dynamically through the master pipeline. The load_run_id is the execution identifier supplied from the pipeline run ID to the notebooks. This separation distinguishes 'which batch' from 'which execution'. Bronze ingestion uses source-file replacement semantics for the same logical batch, while Silver/Gold operate on current state rather than blindly appending results.

Assumption and production enhancement

The assessment does not specify how to distinguish two different versions of a file that reuse the exact same filename. For this exercise, the filename is therefore treated as the batch identifier. If a production source can replace a file's contents under the same name, I would add a content hash, source version, or immutable source batch ID to identify file versions explicitly.

Correction ordering limitation

The supplied source does not provide a reliable record-version number or last-updated timestamp. Therefore the solution does not claim to determine a globally 'latest' version among arbitrary out-of-order arrivals. For the assessment, batches are processed in their supplied arrival sequence and the incoming batch is authoritative for matching Row IDs. In production, I would require a source sequence/version or reliable last-updated timestamp.

5. Data Quality and Validation Policy

The validation design separates structural batch failures from record-level data-quality failures. This prevents a single bad record from unnecessarily discarding an otherwise usable batch, while still protecting reporting from a source structure that no longer matches the expected contract.

Condition	Action	Reason
Missing Row ID	Reject row	Cannot identify or deduplicate the record safely.
Missing Order ID	Reject row	Required transaction context is missing.
Missing Customer ID	Reject row	Cannot reliably associate the transaction with its customer relationship.
Missing Region	Reject row	Regional reporting and region-based security cannot be applied reliably.
Missing Sales	Reject row	Core reporting measure is unavailable; missing is not silently converted to zero.
Sales <= 0	Reject row	Rule defined after profiling the assessment data; negative values are inconsistent with the expected reporting relationship for this exercise.
Unexpected source column / schema drift	Fail batch	The incoming structure no longer conforms to the transformation contract.

Why Batch 3 stops

Batch 3 contains a structural schema change (an unexpected Order Channel field) in addition to record-level quality issues. The schema contract is treated as a batch-level gate. If it fails, no valid rows from that batch are promoted to current-state Silver, Gold is not executed, and the last known good reporting state remains available

Rejection and auditability

Record-level failures are not silently discarded. They are stored in `silver_sales_rejected` with rejection reasons, while `silver_processing_log` records the batch, run, received/passed/rejected counts, status and failure reason. This provides an auditable explanation for both successful and failed processing.

6. Gold Analytical Model

Fact grain

The Gold fact table is at the grain of one current sales record per Row ID. Silver is the authoritative current-state layer; Gold is derived from that current state rather than independently trying to resolve corrections.

Dimensions

The curated model includes date, product and geography dimensions. The geography dimension is at Region–State–City grain. Because Region is repeated across multiple locations, Region alone cannot serve as a unique dimension key. A deterministic geo_key is therefore generated from Region + State + City and stored in both the geography dimension and fact table, enabling a clean one-to-many relationship while retaining Region → State → City analysis.

Gold rebuild strategy

Gold is rebuilt from the authoritative current Silver state rather than blindly appended. This makes the Gold transformation repeatable: the same current Silver state produces the same Gold result. Incremental correction logic belongs in Silver, where Row ID MERGE establishes the current state.

7. Personal Data Protection

The assessment explicitly identifies customer names and postal codes as personal data. The reporting model is designed so regional analytics does not require those identifying fields. Customer name and postal code are therefore excluded from the curated reporting fact used by Power BI. This reduces exposure while retaining the measures and geographic attributes needed for regional sales analysis.

Customer ID may remain as a non-display analytical identifier where required by the model, but identifying customer detail is not exposed in the Gold reporting surface. The key principle is data minimization: only fields necessary for the reporting requirement are exposed to analysts.

8. Row-Level Security

Regional access is driven by the security_user_region mapping table, which maps the authenticated user principal to one or more authorized regions. The SQL security predicate uses USER_NAME() to evaluate the current identity and filter gold_sales_fact accordingly.

Enforcing layer

RLS is enforced at the Fabric SQL data layer through the SQL Analytics Endpoint, not only in Power BI. Power BI therefore consumes already-filtered Gold data. A user querying the protected Gold fact directly through SQL is subject to the same regional predicate and cannot bypass it simply by avoiding the Power BI report.

The geo_key is unrelated to security. It exists for dimensional modeling; security is independently determined by the user-to-region mapping and SQL predicate.

9. Orchestration and Observability

The master pipeline is parameter-driven and coordinates the full flow: Copy to Bronze → Bronze ingestion notebook → Silver validation/current-state processing → Gold transformation. Silver success triggers Gold; Silver failure triggers the failure notification and prevents Gold from executing. The validation failure branch is therefore visible in pipeline run history.

The failure path uses an Office 365 email notification. This satisfies the assessment requirement for a notification or alert on failure. Rejected records and processing logs provide the detailed operational evidence behind the alert.

10. Demonstration Evidence

Demo step	Evidence to show
1. Batch 1	Run batch_01_history.csv; show successful master pipeline; verify Gold/reporting populated.
2. Batch 2	Run batch_02_incremental.csv; show current row count, corrected Row IDs and no duplicate business keys; verify totals reconcile.
3. Batch 3	Run batch_03_incremental.csv; show Silver failure, rejection table/counts, processing log, failure email, and unchanged Gold/reporting state.
4. Batch 2 rerun	Run batch_02_incremental.csv again; show current row count and distinct Row ID count remain unchanged.
RLS	Show the regional Power BI view and a direct SQL query demonstrating that the current identity can access only its authorized region.

11. Scaling to 100 Million Rows per Day

At 100 million rows/day, the first changes would be to make ingestion and processing explicitly partition aware, minimize small files, optimize Delta layout and MERGE strategy, and avoid unnecessary full table rewrites. Batch level parallelism, partition pruning, targeted incremental Gold processing, and appropriate Fabric capacity sizing would become important. I would also introduce stronger metadata/control tables for batch status and source versioning.

The first likely pressure points are Spark startup/compute cost, large MERGE operations on the current-state table, small-file accumulation, and concurrent DirectQuery/reporting load. At that scale I would benchmark the ingestion-to-Silver MERGE path first, then redesign Gold/report serving around incremental aggregates and carefully managed serving tables if the workload required it.

12. What I Would Improve With Another Day

- 1) Introduce an explicit batch-control table with immutable batch IDs, processing status, timestamps and source version/content hash.
- 2) Harden schema management with an explicit versioned schema contract and controlled evolution policy.
- 3) Add richer operational monitoring/metrics and production-grade alert routing.
- 4) Performance-test larger synthetic volumes and tune Delta layout, MERGE strategy and Power BI query behavior.

13. AI and Reference Disclosure

AI assistants and online references were used for architecture discussion, implementation guidance, troubleshooting, documentation drafting and review of Microsoft Fabric concepts. The final solution was implemented and tested in the assessment environment,.